The Half-Life of Context: Measuring and Repairing Context Rot in Large Language Models

Naman Nayak
Chitkara University Institute of Engineering and Technology,
Chitkara University,
Punjab, India
naman0746.be23@chitkara.edu.in

Manveer Singh
Chitkara University Institute of Engineering and Technology,
Chitkara University,
Punjab, India
manveer2063.be23@chitkara.edu.in

Jaiveer Singh
Chitkara University Institute of Engineering and Technology,
Chitkara University,
Punjab, India
jaiveer2104.be23@chitkara.edu.in

Sujal Garg
Chitkara University Institute of Engineering and Technology,
Chitkara University,
Punjab, India
sujal2143.be23@chitkara.edu.in

Abstract: Large language models (LLMs) reason well over short exchanges, but their performance erodes as interactions stretch across many turns. We call this decline *context rot*: the gradual accumulation of stale, contradictory, redundant, and irrelevant material inside the context window. Common fixes (truncation, summarization, compression, retrieval) shrink the context without measuring how quickly its contents decay.
Objectives: We define a *Context Half-Life*, the number of turns after which a piece of information has a 50% chance of no longer helping the model answer, and ask whether it can be measured and used to repair a context.
Methods: We estimate half-lives with survival analysis (Weibull and Kaplan–Meier fits), summarise a session with a Rot Index, and use both to drive a repair loop that UPDATEs outdated facts, MERGEs duplicates, and REMOVEs irrelevant material behind a protection set and verification with rollback. In a deliberately small study on one laptop (one 3-billion-parameter open model, 35 LongMemEval questions, 20 facts per age, synthetic rot-injection sessions) we run five experiments and report every outcome, including those that contradict our seven hypotheses.
Results: Decay depends on content: a changing value fell to 50% correct after 80 turns (Weibull half-life 69 turns, 95% CI 53–102), a stable preference did not decay at all, and a number in a tool output kept 75%. Rewording the question so that it shares no content word with the evidence cut needle-in-a-haystack accuracy from 55% to 30%. Detectors found stale and contradictory segments perfectly on synthetic sessions but flagged irrelevant text with precision of only 38%. Half-life-guided repair cut input tokens by 39% but did not improve accuracy (46% against 54% for full context), raised hallucination, and cost more time than it saved; of seven hypotheses, two were partly supported, three were not supported or could not be tested, and two were refuted at this scale.
Conclusion: Content-dependent decay is measurable even with a 3B model, but at this scale measurement did not translate into better repair. We release code, raw logs, and negative results so that larger studies can build on them.
Keywords: *Context rot, context half-life, long-context reasoning, lost in the middle, needle-in-a-haystack, survival analysis, retrieval-augmented generation, context engineering.*

I. INTRODUCTION

LLMs are increasingly deployed as long-running collaborators: assistants that remember a user across weeks, coding agents that act over hundreds of tool calls, and research agents that read and synthesize dozens of sources. The model keeps no memory between calls, so everything it knows about the task must sit in its *context window*, the text it rereads before every answer. This window is the model's working memory, rewritten continuously as messages, tool outputs, and retrieved documents arrive. Windows now reach hundreds of thousands to millions of tokens, which invites the assumption that capacity implies capability: if the information fits, the model can use it [1].
The evidence says otherwise. Models use information in the middle of a long input less reliably than at its edges [2]; their effective context is often far shorter than claimed [3]; and when question and evidence share few words, 11 of 13 models fall below half of their short-context score at only 32K tokens [4]. Across 18 models, reliability worsens as input grows even on simple tasks [1], and accuracy drops by 13.9% to 85% with length even when all relevant evidence is retrieved perfectly [5]. Conversations suffer too: chat assistants lose about 30% accuracy when recalling information across long histories [6], and splitting single-turn tasks across turns costs 39% on average, mostly through unreliability: once a model takes a wrong turn it rarely recovers [7]. Fig. 1 collects these numbers.

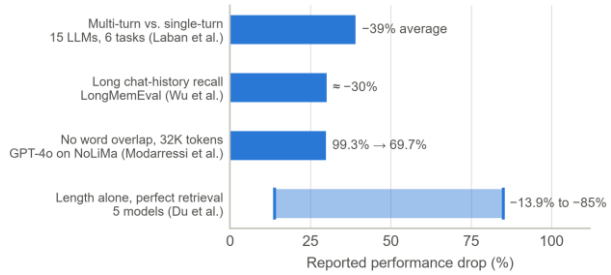


Figure 1: Degradation reported by the studies discussed in Section I. Each number is as reported by its authors; definitions differ, so bars are not directly comparable.

We use *context rot* for the gradual accumulation of stale, contradictory, redundant, and irrelevant material within the window. The term was popularised by the Chroma report [1], and practitioners describe related failures as context poisoning, distraction, confusion, and clash [8]. Length and rot are related but distinct: length is a property of the window, rot a property of its contents. In long-horizon agentic search, degradation depends on what has accumulated, not only on how much [9], while length alone still carries a residual cost [5]. A useful account must therefore separate degradation that is *repairable*, because content has decayed, from degradation that is not. Existing mitigations act on size rather than decay. Prompt compression [10], rolling summarization, retrieval over history [11], and memory layers [12], [13] shrink what the model reads, usually when a token budget is hit, but none estimates how fast each piece of information loses value. Blind shrinking has its own failure modes: summaries drop insights (*brevity bias*) and repeated rewriting erodes detail (*context collapse*) [14], and pruning an agent's history to recent tool calls cut stale-state errors from 47% to 11% of non-completions but doubled premature terminations [15]. Removing the right material requires measuring what has decayed.

We therefore treat context quality as measurable. Our central construct is the **Context Half-Life**: the age, in turns, at which a segment of context has a 50% chance of having lost its value to downstream reasoning, by analogy with the half-life of a radioactive sample or a medicine dose. We contribute:

- **A metric:** Context Half-Life, defined through leave-one-out usefulness and survival analysis, with a session-level Rot Index and degradation-onset estimate.
- **An estimator** for black-box LLMs that combines lightweight detectors, sampled LLM judging, and offline ablation-based calibration.
- **A repair framework** with update, merge, and remove operations, a protection set, and verification with rollback, requiring no retraining.
- **A small, fully reported empirical study:** five experiments with a local 3-billion-parameter model (needle in a haystack, fact ageing, synthetic rot injection, repair on a LongMemEval subset, ablations), with open code and raw logs, including results that contradict our hypotheses.

Four research questions guide the study. **RQ1:** Can the value of contextual information be quantified as a decay process with a stable, predictive half-life? **RQ2:** Does half-life localise degradation (which segments, from which turn) better than context length? **RQ3:** Does half-life-guided repair improve accuracy and hallucination rate over baselines at equal or lower token cost? **RQ4:** Which operations contribute most, and at what latency? An earlier draft of this paper proposed the framework and fixed seven hypotheses before any experiment was run; here we test them at small scale and report each as supported, partly supported, refuted, or not testable. Section II explains how LLMs read context, Section III reviews related work, Section IV presents the method and the hypotheses, Section V the experimental setup, Section VI the results, Section VII the discussion and limitations, and Section VIII concludes.

II. Background: How an LLM Reads Its Context

A. Tokens, attention, and dilution

LLMs read text as *tokens*, frequent words or word pieces; in English one token is roughly three-quarters of a word. The context window must hold the instructions, the whole conversation, any retrieved documents, and the new question, and because the model is stateless the full history is resent on every turn [16]. Transformers [17] decide what matters through *attention*: each token scores the earlier tokens, and the softmax function turns scores into weights that are positive and sum to 1. Attention is therefore a fixed budget shared by everything in the window [18]. If one key token scores s points above n − 1 equal competitors, its share is

$$w(n) = e^{s} / (e^{s} + n - 1) \quad (1)$$

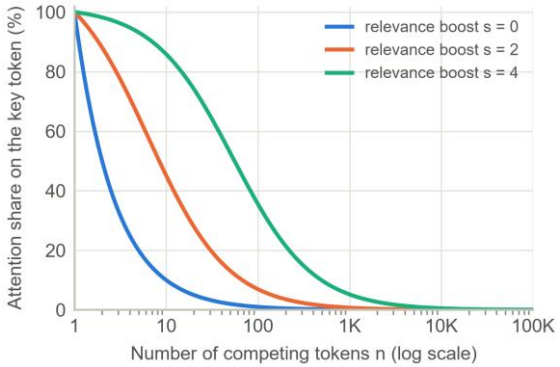


Figure 2: Attention dilution in the toy model of Eq. (1): the key token's share falls as tokens are added, even when it is strongly preferred (calculation, not measured data).

With s = 4, the key token receives about 86% of the attention among 10 tokens but only 5% among 1,000 (Fig.

2). Real models have many heads and layers, so this toy model is an intuition, not a description; still, it shows why one important sentence can be drowned out as the window fills, and why distractors that resemble the answer are especially harmful [1], [19]. Since every token is compared with every other, computation also grows roughly with the square of the input length.

B. Position effects and needle-in-a-haystack tests

Position matters as well as length. In multi-document question answering, accuracy is highest when the relevant document comes first or last and lowest in the middle; for one model a middle-placed answer did worse than no documents at all [2]. The start benefits from *primacy*, since the first tokens attract outsized attention (*attention sinks* [20]), and the end from *recency*. The simplest probe is the needle-in-a-haystack (NIAH) test [21]: hide one out-of-place sentence at a chosen depth in long filler text and ask a question only it answers (Fig. 3). Because the needle and question usually share words, later tests made it harder: RULER adds multi-hop and aggregation tasks [3], NoLiMa removes word overlap, which drops GPT-4o from 99.3% to 69.7% at 32K tokens [4], and the Chroma study varies similarity and distractors [1]. Practical guides follow from these results: put long documents first and the question last, which improved responses by up to 30% in Anthropic's tests [22], and repeat key instructions at both ends [23].

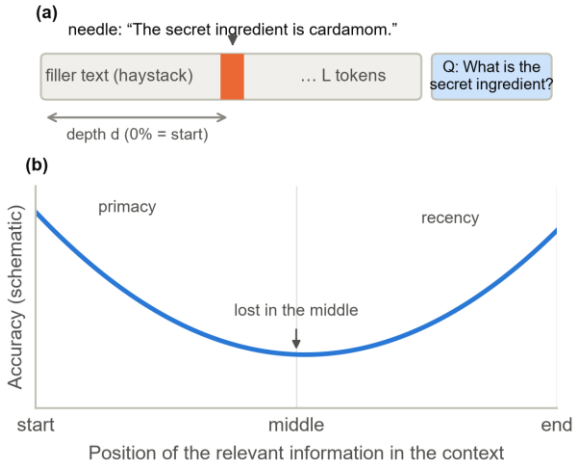


Figure 3: (a) Needle-in-a-haystack setup: a needle at depth d in L tokens of filler, then a question. (b) Schematic of the lost-in-the-middle shape reported by Liu et al. [2] (shape only, no data).

C. Why a half-life?

When something loses a fixed fraction of what remains per unit time, it follows $S(a) = e^{-\lambda a}$ with half-life $H = \ln 2 / \lambda \approx 0.693 / \lambda$. Human memory behaves similarly: Ebbinghaus's forgetting curves fall quickly and then flatten [24]. We ask the same of a context: after how many turns does a piece of information stop helping? Unlike atoms, information has no universal rate (a tool output can be stale within minutes, an allergy matters forever), so we estimate separate half-lives per kind of content.

III. Related Work

A. Degradation in long inputs and long interactions

RULER found that only about half of the evaluated models held up at 32K tokens despite claiming larger windows [3]; NoLiMa showed that earlier robustness partly reflected word matching [4]; the Chroma report found across 18 models that degradation accelerates as question–evidence similarity falls and that haystack structure changes outcomes [1]; and Du et al. showed that length hurts even when irrelevant tokens are replaced by whitespace or masked, proposing recitation of evidence as a mitigation [5]. On LongBench v2, with contexts of 8K to 2M words, the best model answering directly reached 50.1% [25]. In interaction, Laban et al. decomposed the multi-turn drop over 200,000 simulated conversations into a small loss of aptitude and a large rise in unreliability [7]; LongMemEval tests extraction, multi-session and temporal reasoning, knowledge updates, and abstention over histories of about 115K tokens [6]; Xia et al. showed that deleting all history nearly removes rot but leaves many tasks unfinished [9]; and Lodha et al. raised complete task itemization from 71.0% to 91.6% with roughly 63% fewer tokens through pruning and summarization [15]. These studies establish that degradation is real and content-sensitive, but none assigns a decay rate to the content itself.

B. Context management and memory

Breunig's four failure patterns map onto our decay modes: clash and poisoning onto stale and contradictory material, confusion and distraction onto redundant and irrelevant material [8], and models are measurably distracted by irrelevant context [19]. Remedies span token compression (LLMLingua [10]), retrieval-augmented generation [11], and memory systems: MemGPT pages memory like an operating system [12]; generative agents rank memories with an exponential recency decay whose rate is fixed by design, the closest precedent to ours [26]; Mem0 consolidates salient facts, reporting about 91% lower p95 latency and over 90% token savings [13]; Zep marks superseded facts invalid instead of deleting them, gaining up to 18.5% accuracy on LongMemEval [27]; ACE updates context as an evolving playbook to avoid collapse [14]; and AdaCoM learns to manage a frozen agent's context, finding that weaker agents need more aggressive compression [28]. Production guidance adds compaction and the clearing of stale tool results [18]. Table 1 compares these families.

Table 1: Comparison of context-management approaches

Approach	**Typical trigger**	**Decision signal**	**Measures decay explicitly?**
Truncation / sliding window	Token budget	Recency	No
Rolling summarization	Token budget or schedule	LLM judgment of salience	No
Prompt compression (LLMLingua) [10]	Token budget	Token informativeness	No
RAG over history [11]	Every query	Query–passage similarity	No
Memory layers (MemGPT, Mem0) [12], [13]	Write time	Salience, consolidation	No
Temporal graphs (Zep) [27]	Write time	Fact validity intervals	Validity, not value to reasoning
Learned managers (AdaCoM) [28]	Learned policy	Reinforcement-learning reward	Implicit in policy
Context Half-Life (this work)	Measured Rot Index and half-life	Estimated usefulness over time	**Yes**

The gap is a missing measurement: prior systems decide how much to keep, but none estimates how long each piece of information stays useful and uses that estimate to choose among update, merge, and remove. The half-life signal is complementary: memory layers can supply segments, learned managers can execute actions, and the estimate tells them what to act on and when.

IV. Methodology

A. Setting, utility, and decay modes

A frozen *reader* model M answers queries x(t) over T turns. At turn t the context $C(t) = (s_1, \ldots, s_n)$ is a list of *segments* (messages, tool outputs, retrieved passages, memory entries), each with a birth turn b(s), token length $\ell(s)$, a type, and where possible entity–attribute–value triples such as [user, city, Pune]. Answers are scored by $Q(C, x) \in [0, 1]$, for example 1 if correct. Writing C for C(t) and x for x(t), a segment's *utility* is how much quality changes when it is deleted:

$$u(s \mid C, x) = Q(C, x) - Q(C \setminus \{s\}, x) \qquad (2)$$

Positive utility helps, near-zero utility is inert but costs tokens, and negative utility misleads. Because a segment irrelevant now may matter later (a dietary restriction given in turn 3 and needed in turn 80), we average Eq. (2) over a probe set of current and plausible future queries, written u(s, t). Rot takes four forms, shown in Table 2 with a running example: a student, Asha, using a study assistant.

Table 2: Decay modes, how they are detected, and how they are repaired

Decay mode (example)	**Detector**	**Operation**	**Safeguard**
Stale: a later value supersedes it (exam moved from 12 to 19 March)	Entity–attribute–value supersession	UPDATE with change record	Probe the new value; keep history
Contradictory: two claims cannot both hold (assistant said six units; syllabus lists five)	Pairwise entailment plus LLM judge	Resolve by source and recency, else keep both and flag	Protected segments never silently dropped
Redundant: already stated elsewhere (formula sheet pasted three times)	Embedding similarity plus two-way entailment	MERGE	Atomic-fact coverage check
Irrelevant: no expected future use (log of a bug fixed long ago)	Relevance to the current task	REMOVE to archive	Reversible recall

B. Context Half-Life and the Rot Index

Let the age of a segment be $a = t - b(s)$ turns. It *decays* at the first age at which its expected utility falls to a small tolerance ε:

$$D(s) = \min \{ a \geq 0 : u(s, b(s) + a) \leq \varepsilon \} \qquad (3)$$

If a session ends first, the decay time is *right-censored*, which survival analysis handles naturally [29]. The survival function $S(a) = P(D > a)$ is the probability that a segment is still useful at age a, and the Context Half-Life is its median:

$$H = \min \{ a : S(a) \leq 0.5 \} \qquad (4)$$

With a constant decay rate λ, $S(a) = e^{-\lambda a}$ and $H = \ln 2 / \lambda$; for $\lambda = 0.0231$ per turn, $H \approx 30$ turns, so a quarter of such segments survive 60 turns. Because decay need not be constant, we also fit the Weibull form $S(a) = \exp(-(a/\eta)^{\kappa})$ [30], with $H = \eta \cdot (\ln 2)^{1/\kappa}$: $\kappa > 1$ means accelerating decay (volatile tool output), $\kappa < 1$ decelerating decay (durable preferences), as Fig. 4 illustrates. Covariates such as segment type make the half-life conditional, H(z).

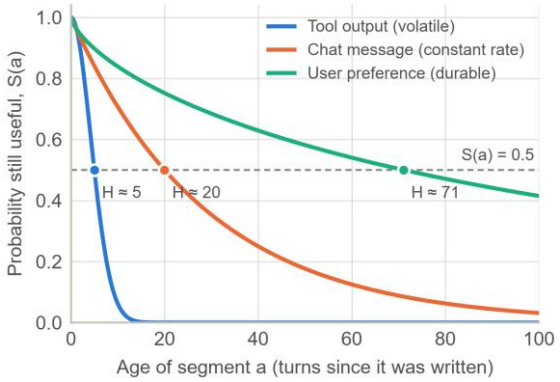


Figure 4: Schematic survival curves for three segment types; dots mark each half-life. Parameters are illustrative: Weibull ($\eta = 6$, $\kappa = 2$), exponential ($H = 20$), and Weibull ($\eta = 120$, $\kappa = 0.7$).

For a whole session, the *live fraction* L(t) and the *Rot Index* R(t) weight each segment's estimated survival $\hat{S}$ by its length, with sums over the segments in C(t):

$$L(t) = \Sigma\, \ell(s) \cdot \hat{S}(t - b(s)) \,/\, \Sigma\, \ell(s) \qquad (5)$$

$$R(t) = 1 - L(t) \qquad (6)$$

R(t) is the expected fraction of the window's tokens that has decayed. For example, a 2,000-token context holding a 200-token system prompt ($\hat{S}$ = 1.0), a 1,200-token old tool output ($\hat{S}$ = 0.2), a 100-token preference ($\hat{S}$ = 0.95), and 500 tokens of recent chat ($\hat{S}$ = 0.6) has L ≈ 0.42 and R ≈ 0.58, and the tool output is the obvious repair target. R(t) splits by decay mode, telling the system what kind of repair is needed. The *session half-life* H(C) is the first turn with L(t) ≤ 0.5, and a change-point fit of accuracy against R(t) and token count gives the onset turn t*. Fig. 5 simulates a 100-turn chat using the curves of Fig. 4: decayed tokens soon dominate, R(t) crosses a repair threshold R* = 0.4 at turn 15, and the session passes its half-life at turn 21.

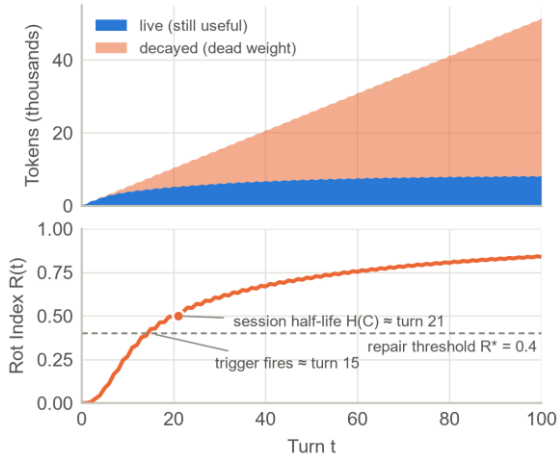


Figure 5: Simulated session (every turn adds 210 chat tokens, every second turn a 600-token tool output, every fifteenth a preference). Top: expected live and decayed tokens. Bottom: Rot Index with the repair trigger and session half-life (model calculation, not measured data).

C. Estimating the half-life for black-box models

Eq. (2) needs one extra reader call per segment and probe, so we estimate in three tiers. **Tier 1 (every turn)** uses cheap detectors: entity–attribute–value extraction flags superseded values; sentence embeddings [31] (vectors whose cosine similarity is near 1 for texts with the same meaning) flag near-duplicates and score relevance to the current task; and an entailment model checks pairs that mention the same entity for contradictions. User-stated facts and preferences are never scored irrelevant by default. **Tier 2** asks a smaller auxiliary LLM to confirm flagged segments with a short rubric, plus a random audit sample to measure detector accuracy. **Tier 3 (offline)** computes leave-one-out utilities with the real reader on development conversations, removing near-duplicate groups together because duplicates hide each other's value. This yields one row per segment per age with a 0/1 decay indicator, on which we fit a discrete-time hazard model, the probability of decaying at age a given survival so far:

$$h(a \mid z) = P(D = a \mid D \geq a, z) = \sigma(w \cdot \varphi(s, C, a)) \quad (7)$$

Here σ is the logistic function and φ holds simple features: type, age, later segments touching the same attribute, contradiction and redundancy scores, task relevance, later references by the assistant, position, and length. This is ordinary logistic regression; survival follows as $\hat{S}(a) = \Pi_{j \leq a}(1 - h(j \mid z))$, and type-level half-lives come from Kaplan–Meier curves [29] with bootstrap intervals [32]. Online cost is O(n) embedding operations per turn plus a few judge calls; ablation is paid once, offline.

D. Half-life-guided context repair

The repair layer sits between the application and the frozen reader and runs the loop of Fig. 6 each turn; no model weights or application logic change. It applies four operations:

- **KEEP** the segment unchanged.
- **UPDATE** a stale segment to the current value while keeping a short change record, since temporal questions may need the history; superseded facts are invalidated rather than deleted [27].
- **MERGE** redundant segments into one canonical segment that must still contain every distinct atomic fact [33], guarding against context collapse [14].
- **REMOVE** irrelevant segments to an indexed archive from which a later query can recall them [11], so removal is reversible.

A **protection set** Π is never edited without verification: instructions, the current goal, user constraints and preferences, unsuperseded personal facts, the last k turns (avoiding the premature terminations caused by aggressive pruning [15]), and anything the assistant recently relied on. **Planning** applies tuned thresholds to the estimated decay-mode probabilities; contradictions are resolved by source and recency (a user's statement outranks an assistant's guess), or both sides are kept with an uncertainty note. Repair is triggered adaptively, when R(t) exceeds R* or the session passes H(C), with a token budget only as a safety net. **Verification** asks probe questions on the original and repaired contexts: unchanged facts must give identical answers and superseded facts the new value; failing operations are rolled back. Fig. 7 applies this to Asha's chat at turn 60, cutting 5,625 tokens to 1,945 (−65%) while probes such as "When is my exam?" (19 March) and "Any allergies?" (peanuts) must still pass.

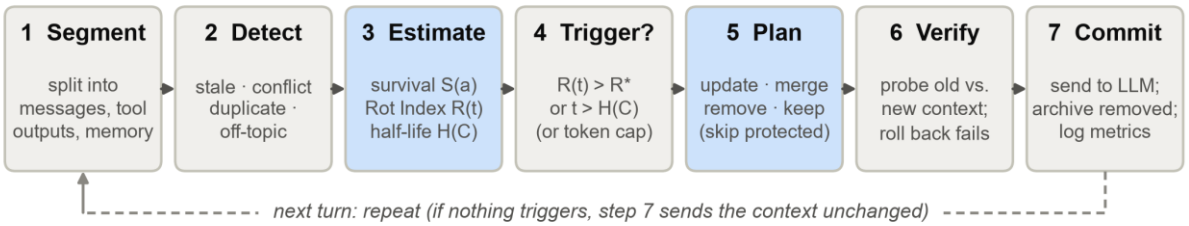


Figure 6: The half-life-guided repair loop, executed once per turn between the application and the frozen LLM. Shaded steps (3 and 5) are where the Context Half-Life estimate drives decisions.

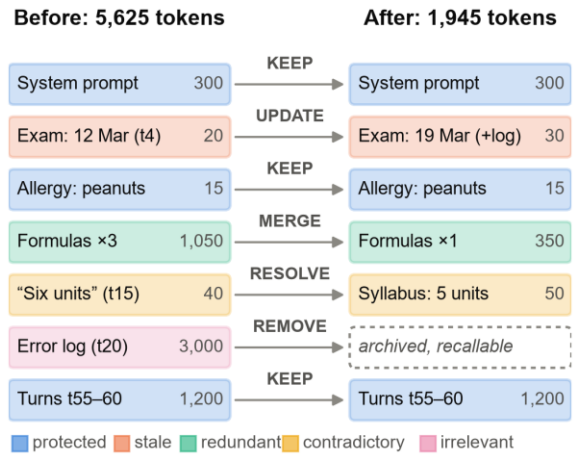


Figure 7: Worked example: repairing a study-assistant context at turn 60. Colours give the diagnosis; the updated exam date keeps a change record, and the error log stays recallable from the archive (illustrative token counts).

E. Reference implementation

The core measurements need only Python, a laptop, and a free local model served by Ollama [16], such as the 3-billion-parameter Llama 3.2; a hosted API can be swapped in by changing one function. A NIAH grid comes first: insert a needle at depths of 0–100% across context lengths, ask the question several times per cell, and plot accuracy as a heat map, then repeat with a paraphrased question and near-miss distractors. Half-life needs *age* rather than position. Listing 1 plants a fact, adds a chosen number of filler turns from any dialogue dataset, and probes; repeating this at several ages with many different facts gives an empirical survival curve.

Listing 1: Fact-ageing probe with a local model

```
def msg(role, text):
    return {"role": role, "content": text}


def build_context(fact, filler, age, rng):
    """Fact first, then `age` filler turns."""
    msgs = [msg("user", fact["statement"]),
            msg("assistant", "Noted!")]
    for u, a in rng.sample(filler, age):
        msgs += [msg("user", u),
                 msg("assistant", a)]
    msgs.append(msg("user", fact["question"]))
    return msgs


def still_useful(chat, fact, filler, age, rng):
    msgs = build_context(fact, filler, age, rng)
    reply = chat.chat(msgs).text.lower()
    return fact["answer"].lower() in reply
```

The survival curve is fitted by maximum likelihood. Listing 2 models the number of successful probes at each age as binomial with success probability $S(a) = p_0 \exp(-(a/\eta)^{\kappa})$; fixing $\kappa = 1$ gives the exponential model with $\lambda = 1/\eta$, and the half-life follows as $H = \eta\,(\ln 2)^{1/\kappa}$. Several starting points guard against local optima, and facts are resampled with replacement for confidence intervals. Repeating the probe for preferences, changing values, and numbers buried in tool outputs yields the type-level half-lives of RQ1.

Listing 2: Estimating λ and the half-life H from probe results

```
def fit_survival(ages, hits, form="weibull"):
    """Binomial MLE of Weibull survival."""
    ages = np.asarray(ages, float)
    hits = np.asarray(hits)
    k = hits.sum(0)
    n = np.full(len(ages), hits.shape[0])
    best = None
    for e0 in (5, 20, 80, 300):
        x0 = [3.0, np.log(e0), 0.0]
        args = (ages, k, n, form)
        r = minimize(_nll, x0, args=args,
                     method="Nelder-Mead")
        if best is None or r.fun < best.fun:
            best = r
    p0 = 1 / (1 + np.exp(-best.x[0]))
    eta = float(np.exp(best.x[1]))
    kappa = 1.0
    if form == "weibull":
        kappa = float(np.exp(best.x[2]))
    return dict(p0=p0, eta=eta, kappa=kappa,
                lam=1 / eta,
                H=half_life(eta, kappa),
                nll=float(best.fun))
```

A small generator then injects the four decay modes at known ages to provide answer keys for the detectors and the onset estimate t*. Repair strategies are plain functions from a segment list to a new list: a sliding window; a rolling summary buffer that condenses the oldest half when a budget is exceeded [18]; a fact card that rewrites changed values and logs old ones (UPDATE); embedding-based de-duplication and relevance pruning (Listing 3); prompt restructuring with documents first, question last, and evidence quoted before answering [5], [22]; and retrieval of archived segments by similarity [11]. The thresholds in Listing 3 are starting points to tune on development data.

Listing 3: Embedding-based MERGE and REMOVE with a protection set

```
def prune(segs, vecs, qvec, protected,
          keep_last=6, dup=0.92, rel=0.20):
    """MERGE duplicates, REMOVE off-topic."""
    cut = max(0, len(segs) - keep_last)
    old, recent = segs[:cut], segs[cut:]
    sim = vecs[:cut] @ vecs[:cut].T
    kept, archive = [], []
    for i, seg in enumerate(old):
        newer = (sim[i][i + 1:] > dup).any()
        offtopic = float(vecs[i] @ qvec) < rel
        if seg["id"] in protected:
            kept.append(seg)          # KEEP
        elif newer or offtopic:
            archive.append(seg)   # MERGE/REMOVE
        else:
            kept.append(seg)
    return kept + recent, archive
```

F. Hypotheses fixed before the experiments

Seven hypotheses were written in an earlier draft of this paper, before any experiment was run; each states what would refute it, and we report every one as supported, partly supported, refuted, or not testable in Section VII.

H1: Decay differs by content type. Tool outputs and changing state should have short half-lives and preferences long ones, with some non-exponential shapes ($\kappa \neq 1$). Refuted if type-level survival curves are indistinguishable or the hazard model performs near chance on held-out data.

H2: Half-life localises degradation better than length. R(t) and H(C) should explain more accuracy decline than token count, and t* should land near injected change points, since rot depends on content [1], [9]; the residual length effect [5] is reported as unrepairable. Refuted if token count dominates R(t) in nested models.

H3: Repair helps most where rot is structural: in LongMemEval's knowledge-update, multi-session, and temporal categories, with a target of +3 percentage points over full context at no more than 70% of its tokens. Refuted if gains are uniform across question types or absent.

H4: Selective repair beats blind shrinking at equal budget, with fewer lost task states, premature terminations [15], and detail losses from rewriting [14]. Refuted if a blinded error analysis shows no difference.

H5: Each operation has a signature: REMOVE gives most token savings, UPDATE most of the knowledge-update gains, MERGE fewer redundancy errors; dropping protection or verification increases regressions.

H6: Hallucination falls without hurting abstention, because stale and contradictory content breeds confident wrong answers. Refuted if repair removes evidence needed to abstain.

H7: Overhead pays for itself: beyond some session length the smaller reader input outweighs detector and judge costs, as memory systems already show [13]. Refuted if no break-even exists within benchmark lengths.

G. What the implementation does differently from the design

The reference implementation simplifies the design above in ways that matter for reading the results. First, the Rot Index used in the experiments is *detector-based*: $\hat{R}(t)$ is the share of tokens in segments flagged by any detector, not the survival-weighted sum of Eq. (6); the survival-weighted form is used only in the illustrative simulation of Fig. 5. Second, the Tier 3 leave-one-out utilities were not computed, so no hazard model of Eq. (7) was fitted; instead, a logistic model of probe failure on content type and age is scored by leave-facts-out AUC. Third, entity–attribute–value extraction is rule-based on the synthetic sessions and done by the 3B model on LongMemEval, and contradiction detection (a natural-language-inference cross-encoder) is used only on the synthetic sessions. Fourth, the REMOVE archive is kept but never recalled, so removal is reversible in principle and not in the measured pipeline.

V. Experimental Setup

A. Hardware, software, and run policy

All experiments ran on one laptop: an Intel Core Ultra 7 255HX CPU, 15.4 GB of RAM, and an NVIDIA GeForce RTX 5050 Laptop GPU with 8 GB of memory, under Windows 11. Models were served by Ollama 0.40.8 [16] (Python client 0.6.3); analysis used Python 3.14.4, NumPy 2.5.4, SciPy 1.18.1, pandas 3.1.0, scikit-learn 1.9.1, and lifelines 0.30.0 [34]; embeddings and the entailment model ran on the CPU through sentence-transformers 6.2.0 and PyTorch 2.14.1. Exact versions are stored with the results. To keep the GPU cool the runner sends one request at a time, pauses briefly after each call, and waits when the GPU passes 78 °C. Reader calls alone totalled about 56 minutes.

B. Models, decoding, and seeds

The reader in every experiment is `llama3.2:3b` (Llama 3.2, 3.2 B parameters, Q4_K_M quantization, digest a80c4f17acd5) [35]; the same model serves as the auxiliary model for attribute extraction and for rolling summaries. The answer judge in E4 is a different model family, `qwen2.5:3b` (3.1 B parameters, Q4_K_M, digest

357c53fb659c) [36]. Decoding is greedy (temperature 0) with a global seed of 20261022, so reruns reproduce every answer on the same software. Sentence embeddings come from all-MiniLM-L6-v2 [31], [37] and contradictions are scored by the cross-encoder nli-deberta-v3-xsmall [38]. A single reference tokenizer (cl100k, via tiktoken) measures all token budgets, so "input tokens" means the same thing in every condition. Larger models, including hosted ones, were not run.

C. Data and subset selection

E1 haystacks are cut from three public-domain novels from Project Gutenberg [39]. E2 draws its filler turns from LongMemEval histories [6] (cleaned release, MIT licence, dataset revision 98d7416), skipping every evidence session, with each message cut to 120 words. The facts of E2 and all sessions of E3 are **synthetic**, generated from templates with seeded random values. E4 uses a stratified random subset of LongMemEval: the plan was 50 questions (12 knowledge-update, 12 temporal, 10 multi-session, 8 single-session, 8 abstention), but only questions whose evidence sessions fit in 4,500 tokens qualify, because each haystack is padded with the question's own distractor sessions to about 8,000 tokens in date order. That left 35 questions: 2 knowledge-update, 12 temporal, 7 multi-session, 8 single-session, and 6 abstention.

D. Protocols

E1 (needle in a haystack). Five lengths (1K to 16K tokens), eleven depths (0–100%), three needles, and two wordings give 330 calls. The literal question reuses the needle's words; the paraphrase shares no content word with it, which the code checks. Correctness is a normalised string match on the answer.

E2 (fact-ageing half-life). Three synthetic fact types: a stable preference (a project codename), a changing value (a city, corrected right after it is first stated, so age counts turns since the correction), and a number buried in a roughly 250-token tool output. Each fact is probed at ages 0, 5, 10, 20, 40, and 80 turns of filler, with 20 facts per type; filler is nested, so the age-5 context is a prefix of the age-10 context. A second, *length-matched* run (10 facts per type) fixes the total at 80 filler turns and varies only how many turns follow the fact, separating age from length. Per type we fit exponential and Weibull survival curves by binomial maximum likelihood, bootstrap facts (500 resamples) for confidence intervals, draw Kaplan–Meier curves from each fact's first failure [29], compare types with a log-rank test, test $\kappa = 1$ with a likelihood-ratio test, and score a logistic model of failure on type and age by leave-facts-out AUC.

E3 (rot injection, synthetic). We generate 40 sessions of 40 turns with known labels: three attributes that change value (the old statement is *stale*), two verbatim re-pastes (the older copy is *redundant*), three build logs (*irrelevant*), and one wrong assistant claim (*contradictory*). Eight sessions are a development split on which the duplicate and relevance thresholds are tuned; the other 32 are the test split. Detectors see only text. $\hat{R}(t)$ is computed on the prefix of each session, and the onset t^* is the least-squares mean-shift change point of that series. A reader probes each attribute's current value at ten checkpoints (every four turns); a *twin* session with the same turn structure but every injected segment replaced by neutral filler is probed at the end.

E4 and E5 (repair comparison and ablations). For each question the haystack is rendered as a transcript and the reader answers once per condition. Conditions: full context (B0), sliding window (B1), rolling summary (B2), random and oldest-first removal, half-life-guided repair (HL), and an oracle that sees only the evidence sessions. HL extracts user-stated attributes with the 3B model from messages that contain a change cue (a time word or a digit), canonicalises attribute names by embedding similarity (cosine ≥ 0.85), flags stale values, near-duplicates (cosine above 0.95), and segments with relevance to the question below 0.05 (both thresholds tuned on E3's development split), protects user-stated values and the last six segments, and repairs when $\hat{R}$ exceeds 0.10 (fixed in advance, not tuned). Verification re-asks the question and rolls the repair back if the answer turns into an abstention that the full context did not give; no gold label is used. B1, B2, random, and oldest-first receive, for each question, the token budget that HL ended up with. E5 re-runs HL without UPDATE, without MERGE, without REMOVE, without the protection set, without verification, and with a token-budget trigger (repair only above 90% of the target length). Identical prompts are answered once and the answer is reused.

E. Metrics and statistics

Table 3 defines the metrics. Hallucination is a confident wrong answer: an incorrect answer that is not an abstention, or any non-abstaining answer to an unanswerable question. Correctness is judged by the second model with LongMemEval-style prompts [6]. Comparisons are paired by question: exact McNemar test [40], paired bootstrap intervals with 10,000 resamples [32], and Holm's correction [41] over the five comparisons of HL against the other strategies. Intervals are percentile bootstrap intervals over questions or facts, and with samples this small they are wide.

Table 3: Evaluation metrics (reasoning-quality rubric and human audit were not run)

Dimension	How it is computed
Task accuracy	Correct answers ÷ questions (LLM judge; string match for E1–E3)
Hallucination rate	Share of confident wrong answers, plus any answer to an unanswerable question
Token efficiency	Input tokens per query; accuracy per 1,000 tokens
Latency	Median and 95th-percentile reader time; repair overhead reported separately
Context health	Type-level half-life H, Rot Index $\hat{R}(t)$, onset turn t^*

F. Deviations from the planned study

Because the study had to fit about two hours on one laptop, it is much smaller than the design in the earlier draft. Section VII lists the consequences; here we state the facts.

- One 3B reader and a 3B judge instead of two open-weight and two proprietary readers; the judge agrees with plain string matching on only 77% of 377 answerable-question answers.
- 35 questions with about 8K-token contexts instead of 100 questions with contexts of up to 115K tokens; LongBench v2, NoLiMa, and the Chroma toolkit were not run.
- No LLMLingua, memory-layer, or learned-manager baselines; no reasoning-quality rubric; no human audit of the judge; no blinded error analysis (H4).
- The Rot Index is detector-based, and no leave-one-out utilities or hazard model were computed (Section IV-G).
- E3 is not a clean held-out test. While debugging, we ran the detectors on the whole synthetic set, found that the first generator reused ten filler sentences (so the filler itself was redundant) and that the value extractor mishandled titles such as "Dr.", fixed both, and re-ran everything before reporting; only the tuned thresholds were restricted to the development sessions.

G. Code and data availability

All code, raw per-call logs (JSON Lines), summary tables, and the build scripts of this paper are in a public repository [42]. Every number in Section VI is read from a file under `results/` when the paper is built, and the build fails if a quoted number is missing.

VI. Results

A. E1: needle in a haystack

Accuracy with the literal question is 55% overall (95% CI 47–62) and falls from 73% at 2K tokens to 42% at 8K and 45% at 16K (Fig. 8). Rewording the question so that it shares no content word with the needle lowers overall accuracy to 30%, a paired difference of −24.8 points (95% CI −33.3 to −16.4; McNemar $p < 0.001$); at 16K tokens the paraphrased question is answered correctly only 18% of the time. The famous lost-in-the-middle dip [2] does not appear: literal accuracy at depths 30–70% is 63%, against 50% at depths of at most 10% or at least 90%. With three needles per cell, each heat-map cell can only take the values 0, 33, 67, and 100%, so single cells should not be interpreted; the trends by length and wording are what the data support.

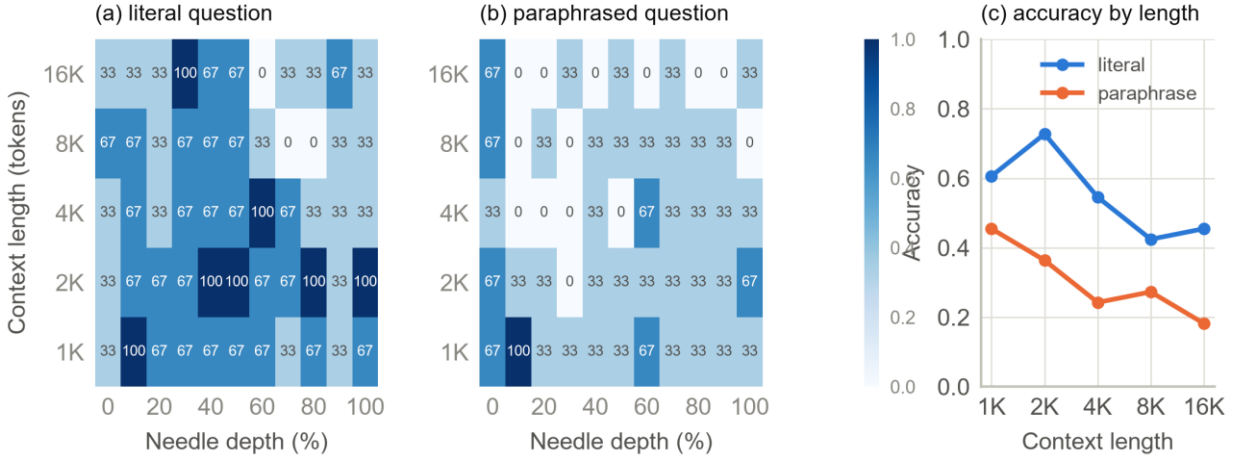


Figure 8: E1 (measured, llama3.2:3b). (a, b) Accuracy (%) by context length and needle depth for the literal and the paraphrased question (3 needles per cell). (c) Accuracy by length.

B. E2: fact-ageing half-life

Table 4 and Fig. 9 show that content type matters. A *changing value* is answered correctly at every age up to 20 turns, then drops to 55% at 40 turns and 50% at 80; its Weibull half-life is 69 turns with shape $\kappa = 1.77$ (95% CI 1.38–2.32), meaning accelerating decay, and the likelihood-ratio test rejects a constant rate ($p = 0.03$). The mechanism is staleness: at 80 turns the model answered with the *old* city in 50% of the probes. A *stable preference* never decayed within 80 turns, so its half-life is right-

censored (above 80), and a *number in a tool output* was still found in 75% of probes at 80 turns; its fitted half-life lies beyond the measured horizon, and the very large κ simply reflects a single step between 40 and 80 turns, so the narrow interval for the tool-output Weibull fit should not be read as precision. The log-rank test separates the three types ($p < 0.001$), and a logistic model of failure on type and age reaches a leave-facts-out AUC of 0.94.

Table 4: E2 survival fits, ageing design (20 facts per type; 95% bootstrap CIs in parentheses)

Fact type	S(80)	Weibull H (turns)	κ	Exp. H (turns)
Stable preference	100%	> 80 (censored)	–	–
Changing value	50%	69 (53–102)	1.8 (1.4–2.3)	90 (61–157)
Number in tool output	75%	beyond horizon[a]	poorly identified[a]	401 (210–2121)[a]

[a] Extrapolated beyond the 80-turn horizon from a single step in the data. The exponential half-life of the preference is not estimable because no failure occurred.

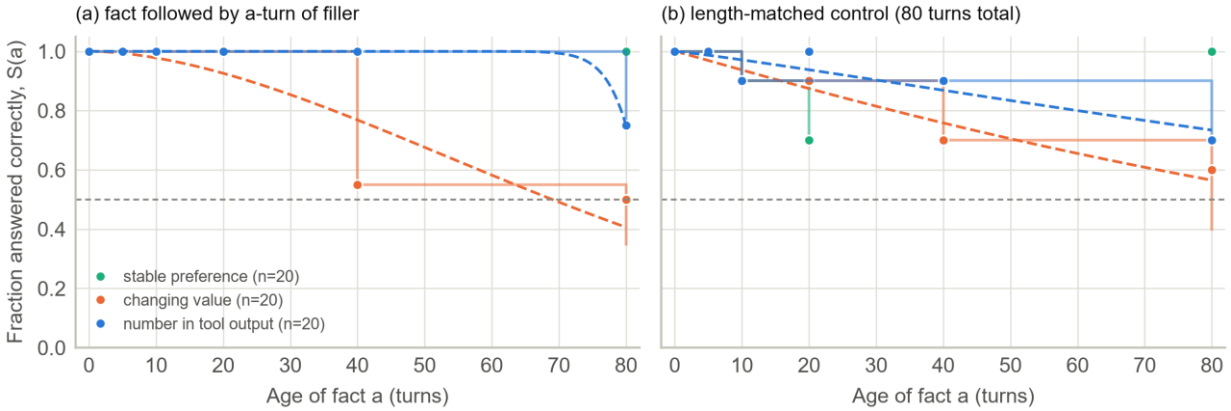


Figure 9: E2 (measured, synthetic facts). Fraction of facts answered correctly by age. Dots: observed; faint steps: Kaplan–Meier curves from first failure; dashed: Weibull fits (not drawn for the censored preference). (a) Filler follows the fact (20 facts per type). (b) Length-matched control (10 facts per type).

The length-matched control (Fig. 9b) does *not* reproduce this separation: with the total length fixed at 80 turns and only 10 facts per type, the log-rank test finds no difference between types ($p = 0.39$) and the stable preference dips to 70% at age 20 before recovering, which is sampling noise at this sample size. Age and length are therefore confounded in the main design, and the half-lives above describe decay in a growing context, not age alone.

C. E3: rot injection (synthetic sessions)

Table 5 reports detector quality on the 32 test sessions. Because the stale and contradictory statements follow fixed templates, rule-based attribute extraction plus the entailment model find them perfectly, which says more about the generator than about real conversations. The redundancy detector has full recall but precision of only 82%, and the relevance detector flags 237 segments for 93 true irrelevant ones (precision 38%), because study-topic chat is also distant from the current question.

Table 5: E3 detector quality on synthetic test sessions (95% CI over sessions)

Mode	True	Flagged	Precision (%)	Recall (%)
Stale	96	96	100 (100–100)	100 (100–100)
Contradictory	32	32	100 (100–100)	100 (100–100)
Redundant	64	83	82 (76–89)	100 (100–100)
Irrelevant	93	237	38 (34–41)	100 (100–100)

The onset estimate recovers the injected change point only weakly (Fig. 10b). The mean absolute error is 8.7 turns (95% CI 7.2–10.2), against 11.2 for a guess drawn uniformly over the session; the estimate is biased late by 7.8 turns on average, only 31% of sessions fall within five turns, and the correlation between true and estimated onset is 0.46. The onset found from reader accuracy is no better (mean absolute error 8.2 turns), because reader accuracy is noisy and flat (Fig. 10c). The injected rot also barely hurt the reader: at the end of the session, accuracy was 66% with rot and 71% in the rot-free twin, a difference of +5.2 points (95% CI −5.2 to +15.6; $p = 0.42$). Adding $\hat{R}$ to a logistic model that already contains the token count improves the fit only marginally (likelihood-ratio $p = 0.07$).

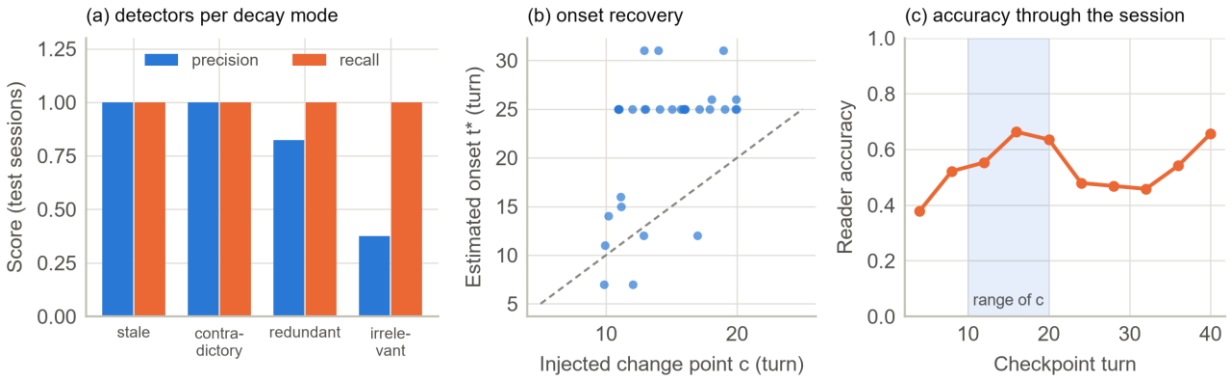


Figure 10: E3 (measured, synthetic sessions). (a) Detector precision and recall by decay mode. (b) Estimated onset t* against the injected change point c (dashed line: perfect recovery; points jittered horizontally). (c) Reader accuracy at each checkpoint turn; the shaded band is the range of c.

D. E4: repair comparison

Table 6 and Fig. 11 compare the strategies on 35 questions. HL repair fired on 100% of questions (mean $\hat{R}$ = 0.45) and cut the input from 7,983 to 4,908 tokens, but accuracy fell from 54% to 46% and hallucination rose from 31% to 43%. None of the paired differences against HL is significant after Holm's correction (Table 7); the point estimate against full context is −8.6 points, HL ties the sliding window, random removal, and oldest-first removal, and its edge over the rolling summary is +5.7 points. In short, HL is about as good as other ways of shrinking the context by 39%, and none of them matched full context. The oracle, which sees only the evidence sessions, scores 54%, equal to full context, so the reader's own ability, not the distractors, limits accuracy on this subset.

Table 6: E4 main comparison (95% bootstrap CIs; reader latency in seconds)

Condition	Accuracy (%)	Halluc. (%)	Input tokens	Acc. per 1K tok.	p50 (s)	p95 (s)
B0 full context	54 (37–71)	31	7,983	0.068	3.1	3.5
B1 sliding window	46 (29–63)	29	4,695	0.097	2.0	2.6
B2 rolling summary	40 (23–57)	26	3,861	0.104	1.5	2.0
Random removal	46 (29–63)	26	4,663	0.098	1.8	2.2
Oldest first	46 (29–63)	29	4,695	0.097	2.0	2.6
HL repair (this work)	46 (29–63)	43	4,908	0.093	1.8	2.6
Oracle (evidence only)	54 (37–71)	31	3,470	0.156	1.3	1.9

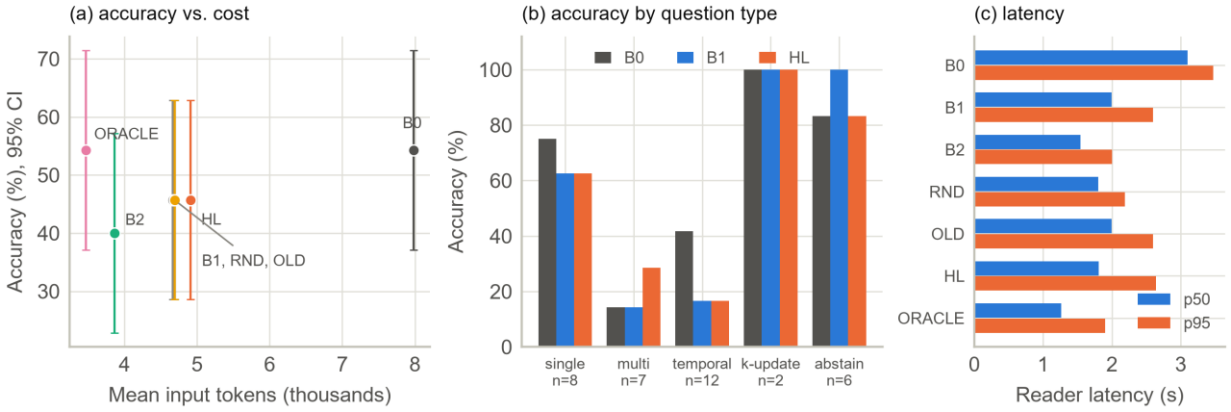


Figure 11: E4 (measured, 35 LongMemEval questions). (a) Accuracy against mean input tokens with 95% CIs; B1, random, and oldest-first coincide. (b) Accuracy by question type with the number of questions. (c) Median and 95th-percentile reader latency.

Table 7: HL against each strategy: paired difference in accuracy

Comparison	Diff. (points)	Holm p
HL − B0	−8.6 (−22.9 to +2.9)	1.00
HL − B1	+0.0 (−11.4 to +11.4)	1.00
HL − B2	+5.7 (−11.4 to +22.9)	1.00
HL − Random	+0.0 (−14.3 to +14.3)	1.00
HL − Oldest	+0.0 (−11.4 to +11.4)	1.00

By question type (Fig. 11b), the picture is mixed and every cell is tiny. On the 12 temporal questions HL scores 17% against 42% for full context, the clearest loss; on the 7 multi-session questions it scores 29% against 14%; on abstention it ties full context at 83%. Only 2 knowledge-update questions fit the length cap, and all strategies answered both, so the case the method is designed for was not tested. The stale-value detector found 0.0 stale segments per question, so UPDATE never fired on real conversations; the detector flagged 0.3 redundant and 18.8 irrelevant segments per question, so nearly all of the token saving comes from REMOVE.

Repair is slow in this implementation. The reader needs 1.9 s on the repaired context against 3.0 s on the full one, a saving of 1.2 s, but detection and repair on the CPU take 6.0 s per question, so the total is 7.9 s.

E. E5: ablations

Table 8 removes one component at a time. Without UPDATE, without MERGE, without the protection set, without verification, and with a token-budget trigger, accuracy equals that of full HL, because UPDATE and MERGE almost never fired, the protection set and verification rarely mattered (verification rolled back 2.9% of questions, one question in 35), and the token trigger fired whenever $\hat{R}$ did. Only removing REMOVE changes anything: accuracy rises to 57% while the input stays at 7,948 tokens, which confirms that REMOVE alone delivers the savings and also the losses.

Table 8: E5 ablations of HL repair (paired difference against full HL)

Variant	Acc. (%)	Diff. vs HL (points)	Tok.	Hall. (%)
Full HL	46	reference	4,908	43
− UPDATE	46	+0.0 (+0.0 to +0.0)	4,908	43
− MERGE	46	+0.0 (+0.0 to +0.0)	4,921	43
− REMOVE	57	+11.4 (+0.0 to +25.7)	7,948	29
− protection	46	+0.0 (+0.0 to +0.0)	4,905	43
− verification	46	+0.0 (+0.0 to +0.0)	4,908	43
Token trigger	46	+0.0 (+0.0 to +0.0)	4,908	43

VII. Discussion

A. Verdict on each hypothesis

Table 9 gives the verdict on each of the seven hypotheses of Section IV-F, with the evidence that decides it. Two are partly supported, three are not supported or could not be tested, and two are refuted.

Table 9: Verdict on the seven hypotheses

H	Verdict	Evidence
H1	Partly supported	Types separate in the ageing design (log-rank $p < 0.001$; AUC 0.94; changing $\kappa = 1.8$) but not in the length-matched control ($p = 0.39$); n is small.
H2	Not supported	Onset error 8.7 turns vs 11.2 for a uniform guess; $\hat{R}$ adds little to token count ($p = 0.07$); rot barely hurt the reader.
H3	Not supported; mostly not testable	2 knowledge-update questions; HL −8.6 points vs full context overall.
H4	Not supported	HL ties B1, random, and oldest-first; a +5.7-point edge over B2 is not significant; no error analysis was run.
H5	Partly supported	REMOVE alone gives the token savings; UPDATE and MERGE never fired, so their signatures could not be tested.
H6	Refuted	Hallucination 31% → 43%; abstention accuracy unchanged (83%).
H7	Refuted at 8K tokens	Total time 7.9 s vs 3.0 s; no break-even in the lengths measured.

B. What the results do and do not show

The strongest finding is empirical and supports the premise: information decays at different rates depending on what it is, and the decay can be measured with ordinary survival tools even on a 3B model. The changing value, the case that is stale by construction, decays with an increasing hazard, and the model's mistakes are the old value rather than random errors; a stable preference does not decay at all. This pattern is what the half-life idea predicts. The needle-in-a-haystack result adds that retrieval depends heavily on question wording, in line with earlier reports that matching words inflate long-context scores [4], while the position effect reported for larger models [2] is not visible for this one.

The weakest finding is the second half of the thesis: using the measurement to repair the context did not help. Several explanations are plausible, and our data do not separate them. The reader is small, and the oracle result shows that it fails on about half of the questions even when given only the evidence, so there is little headroom for any context repair. The judge is a 3B model that agrees with string matching on about three answers in four, so accuracy differences of a few points are within its noise. The relevance detector is crude: it uses sentence-embedding similarity to the question, which removes evidence for multi-hop and temporal questions whose evidence does not resemble the question, and it fired on every question. The stale-value detector depends on the same small model and found nothing. Finally, all contexts were about 8K tokens, a length at which a 3B model is not yet overwhelmed, so the potential gain from shrinking was small, and the embedding and extraction steps were slow on a laptop CPU. None of these is evidence that half-life-guided repair cannot work; each is a reason the present test was weak.

The synthetic E3 results should be read with caution. They show that the machinery works when the rot is cleanly templated, but the generator and the detectors were

written by the same authors, the detectors' thresholds were tuned on the same generator, and the one measurable effect we hoped for, a drop in reader accuracy after the change point, did not appear at this scale.

C. Implications

Three practical points follow. First, measure staleness separately from length: content-dependent decay appeared even with a short, constant context, and a fixed truncation rule would discard stable preferences and changing values alike. Second, any context-repair method needs a ceiling check such as our oracle condition; here it showed immediately that the reader, not the context, was the bottleneck. Third, repair that costs more compute than the reader time it saves needs long contexts or cheaper detectors to pay off, so detector cost belongs in the evaluation, not just in the discussion.

D. Threats to validity and limitations

- **Scale and power:** one 3B model, one seed, 20 facts per age, and 35 questions give wide intervals; differences of ten points are not detectable. Failure to find an effect is not evidence of no effect.
- **Confounding:** in E2 age and length grow together; the length-matched control is half the size and did not replicate the separation of types.
- **Synthetic data:** E2 facts and all E3 sessions are template-generated, the detectors read the same templates, and real conversations are messier.
- **Evaluation:** the judge is a weak, same-scale model with 77% agreement with string matching [43]; LongMemEval items may be contaminated; no human audit was done.
- **Coverage:** hosted and larger models, LongBench v2, NoLiMa, the Chroma toolkit, and other memory systems were not evaluated, and the knowledge-update stratum was almost empty.
- **Systems and security:** editing early context can invalidate prompt caches, and a repair layer is itself an attack surface, since a poisoned tool result could trigger removal of real constraints.

VIII. Conclusion and Future Scope

We proposed to measure how quickly the contents of an LLM context lose value and to repair the context accordingly. In a small, fully reported study with a 3B open model we found that decay does depend on content type, that a changing value loses usefulness with an accelerating hazard while a stable preference does not, and that question wording matters more than context length. We did not find that the measurement improves repair: half-life-guided repair saved about 39% of the tokens, did not raise accuracy, increased hallucination, and cost more time than it saved. Two of seven hypotheses were partly supported, three were not supported or could not be tested, and two were refuted at this scale.

The next steps are to test the same protocol where the claim is most likely to hold: readers of 8B parameters and above and a hosted model [35]; a larger knowledge-update subset with an entailment-based staleness detector instead of small-model extraction; contexts of 32K tokens and more, where shrinking should matter and where LongBench v2 and NoLiMa apply; GPU-side embeddings to reduce overhead; recall from the archive; learned hazard and repair policies [28]; a human audit of the judge; and a dedicated decay benchmark with ground-truth half-lives, compared with human forgetting curves [24]. Cache-aware repair scheduling and defences against poisoned tool outputs remain open. The code and every raw log are public so that these studies can start from our negative results rather than repeat them.

References:

[1] K. Hong, A. Troynikov, and J. Huber, "Context rot: How increasing input tokens impacts LLM performance," Chroma, Tech. Rep., Jul. 2025. [Online]. Available: https://research.trychroma.com/context-rot

[2] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[3] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?" in *Proc. Conf. Language Modeling (COLM)*, 2024, arXiv:2404.06654.

[4] A. Modarressi, H. Deilamsalehy, F. Dernoncourt, T. Bui, R. A. Rossi, S. Yoon, and H. Schütze, "NoLiMa: Long-context evaluation beyond literal matching," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, PMLR, vol. 267, 2025, pp. 44554–44570.

[5] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. B. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng, "Context length alone hurts LLM performance despite perfect retrieval," in *Findings Assoc. Comput. Linguistics: EMNLP 2025*, 2025, arXiv:2510.05381.

[6] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025, arXiv:2410.10813.

[7] P. Laban, H. Hayashi, Y. Zhou, and J. Neville, "LLMs get lost in multi-turn conversation," arXiv:2505.06120, 2025.

[8] D. Breunig, "How to fix your context," dbreunig.com, Jun. 2025. [Online]. Available: https://www.dbreunig.com/2025/06/26/how-to-fix-your-context.html

[9] S. Xia, Y. Wang, Z. Huang, and P. Liu, "Diagnosing and mitigating context rot in long-horizon search," arXiv:2606.29718, 2026.

[10] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2310.05736.

[11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[12] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.

[13] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," arXiv:2504.19413, 2025.

[14] Q. Zhang et al., "Agentic context engineering: Evolving contexts for self-improving language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026, arXiv:2510.04618.

[15] A. Lodha, M. Pahlavikhah Varnosfaderani, A. Chakraborty, and A. Mithal, "Less context, better agents: Efficient context engineering for long-horizon tool-using LLM agents," arXiv:2606.10209, 2026.

[16] Ollama, "Ollama Python library," GitHub repository. [Online]. Available: https://github.com/ollama/ollama-python

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[18] Anthropic, "Effective context engineering for AI agents," Anthropic Engineering Blog, 2025. [Online]. Available: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[19] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.

[20] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2309.17453.

[21] G. Kamradt, "Needle in a haystack: Pressure testing LLMs," GitHub repository, 2023. [Online]. Available: https://github.com/gkamradt/LLMTest_NeedleInAHaystack

[22] Anthropic, "Long context prompting tips," Claude Developer Platform Documentation. [Online]. Available: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/long-context-tips

[23] OpenAI, "GPT-4.1 prompting guide," OpenAI Cookbook, Apr. 2025. [Online]. Available: https://cookbook.openai.com/examples/gpt4-1_prompting_guide

[24] H. Ebbinghaus, *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Leipzig, Germany: Duncker & Humblot, 1885.

[25] Y. Bai et al., "LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2025, doi: 10.18653/v1/2025.acl-long.183.

[26] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST)*, 2023.

[27] P. Rasmussen et al., "Zep: A temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025.

[28] L. Yi et al., "Learning agent-compatible context management for long-horizon tasks," arXiv:2605.30785, 2026.

[29] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *J. Amer. Statist. Assoc.*, vol. 53, no. 282, pp. 457–481, 1958.

[30] W. Weibull, "A statistical distribution function of wide applicability," *J. Appl. Mech.*, vol. 18, pp. 293–297, 1951.

[31] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[32] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[33] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[34] C. Davidson-Pilon, "lifelines: Survival analysis in Python," *J. Open Source Softw.*, vol. 4, no. 40, p. 1317, 2019.

[35] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[36] A. Yang et al., "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[37] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[38] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023, arXiv:2111.09543.

[39] Project Gutenberg, "Free eBooks." [Online]. Available: https://www.gutenberg.org

[40] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[41] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[42] N. Nayak, M. Singh, J. Singh, and S. Garg, "context-half-life: Code, raw logs, and paper source," GitHub repository, 2026. [Online]. Available: https://github.com/Namann-14/context-half-life

[43] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, Datasets and Benchmarks Track, 2023.